Novel Multi-Corner Delay Padding using Path Relationship Analysis and Dual Decomposition

Wai-Shing Luk

October 13, 2026

Abstract

Delay padding inserts extra delay into selected data paths to repair the hold violations that clock skew scheduling cannot, but applying it consistently across process, voltage, and temperature corners remains hard. We present a multi-corner delay padding methodology built on two ideas. First, path relationship analysis (PRA) decides *where* delay may physically be inserted before deciding *how much* to insert: the maximum-delay (setup) and minimum-delay (hold) paths of every register pair are classified into four structural configurations, and the timing constraint graph is augmented with auxiliary nodes and coupling edges to encode them. Second, a dual decomposition coordinates the per-corner padding subproblems—each a network flow problem—through shared delay variables and Lagrange multipliers, which enforces a single physical padding assignment and removes the oscillatory "ping-pong" effect of corner-by-corner optimization. Statistical robustness is provided by modeling path delays with the generalized extreme value (GEV) distribution, whose quantile function converts probabilistic timing constraints into deterministic difference constraints. The resulting flow couples a binary search on the clock period with a negative-cycle feasibility test and the dual decomposition solver. On the OpenCore and OpenABC-D benchmarks with the ASAP7 7,nm library across the FF, SS, and TT corners at a 0.99 timing yield, the method reduces the optimized clock period by an average of 6.12% relative to the single worst-case (SS) corner, with a largest gain of 9.22%.

1 Introduction

Modern synchronous integrated circuits must function correctly across a wide range of operating conditions. Process, voltage, and temperature (PVT) variations, together with manufacturing uncertainty, cause the delays of gates and interconnects to differ from their nominal values, so a design that meets its timing specification in one corner may fail in another. Guaranteeing both correctness and robustness across these conditions—*timing closure*—is one of the most demanding tasks in physical design.

Two constraints govern correct data transfer between flip-flops (FFs). The *setup* constraint requires that data launched from a register arrive at the capturing register early enough to be safely sampled before the active clock edge. The *hold* constraint requires that the same data remain stable long enough after the edge that it is not captured prematurely. Violating either constraint causes a setup or hold failure and, ultimately, a malfunctioning circuit.

Conventional timing closure optimizes a design for a *single worst-case corner*. This is both pessimistic—it discards performance achievable under typical conditions—and, in a multi-corner setting, unstable. A padding or sizing decision that removes a violation in a slow corner may introduce one in a fast corner, and the subsequent correction may reintroduce the original violation. The result is the well-known "ping-pong" effect: an oscillation between corner-specific fixes that need not converge.

Clock skew scheduling (CSS) is the classical remedy that avoids touching the data paths [1], [2], [3]. By deliberately adjusting the arrival times of the clock at individual FFs, CSS balances setup and hold constraints and recovers otherwise unusable slack. Its constraints form a system of difference constraints, which is feasible if and only if the associated *timing constraint graph* (TCG) contains no negative cycle. When a negative

cycle exists, however, no clock skew assignment can satisfy all constraints, and the data paths themselves must be modified.

Delay padding changes those paths by inserting additional delay, normally by substituting a slower cell for a faster one, to relax hold constraints without altering logic functionality [4]. Combined with CSS, it provides an additional degree of freedom and substantially enlarges the space of timing-feasible designs. Closely related network formulations minimize the cycle time or maximize the slack of a chip [5] and co-optimize the schedule with clock-tree construction [6].

The conventional formulation of delay padding minimizes the total inserted delay $\sum_e p_e$, which is the dual of a minimum-cost flow problem and admits elegant network algorithms [7]. In a modern standard-cell flow this objective is a poor proxy for design cost: delay is inserted by cell substitution rather than by an abstract resource, so the *amount* of delay is not what the designer pays for. Two further difficulties compound the problem. First, *physical feasibility*: some paths offer no valid position at which delay can be inserted, so a solution that merely minimizes $\sum_e p_e$ may be unrealizable. Second, conventional methods ignore the structural *relationship* between the maximum-delay (setup) path and the minimum-delay (hold) path of a register pair—the two paths may share logic, be disjoint, or be nested—yet that relationship determines whether padding one path silently perturbs the other.

This paper presents a methodology that addresses all three difficulties and, in addition, coordinates padding across corners. Its contributions are as follows.

1. **Path relationship analysis (PRA).** We invert the conventional order of decisions: instead of computing required delays and then searching for positions, we first identify *physically feasible* padding positions and then compute their values. The maximum- and minimum-delay paths of each register pair are classified into four structural configurations, and the TCG is augmented with auxiliary nodes and coupling edges that encode the resulting constraints ($p_s = p_h$ or $p_s \geq p_h$).
2. **Dual decomposition for multi-corner consistency.** We decompose the multi-corner problem into one network-flow subproblem per corner and couple them through shared delay variables and Lagrange multipliers. The coordination enforces a single physical padding assignment across all corners and removes the ping-pong effect by construction.
3. **GEV-based statistical timing.** We model path delays with the generalized extreme value (GEV) distribution, whose heavy tail matches the observed behavior of maximum path delays better than a Gaussian model. Its quantile function converts probabilistic timing constraints into deterministic difference constraints at a prescribed timing yield β.
4. **An integrated flow and its evaluation.** We combine a binary search on the clock period, a negative-cycle feasibility test, and the dual decomposition solver, and evaluate the method on the OpenCore and OpenABC-D benchmarks with the ASAP7 7,nm predictive library [8] across the FF, SS, and TT corners at a yield target $\beta = 0.99$. The method reduces the optimized clock period by an average of 6.12% relative to the single worst-case (SS) corner.

The remainder of the paper is organized as follows. Section 2 fixes notation and recalls the timing constraints and the TCG. Section 3 introduces the GEV statistical delay model and the quantile-based timing constraints. Section 4 develops path relationship analysis and the four structural configurations. Section 5 presents the dual decomposition for multi-corner padding. Section 6 assembles the complete optimization flow. Section 7 reports the experimental setup and results, and Section 8 concludes.

2 Background: Timing Constraints and the Timing Constraint Graph

2.1 Clock skew and timing constraints

Let t_i denote the clock arrival time at flip-flop FF_i. For a data path from FF_i to FF_j, the *clock skew* is the difference of the two arrival times,

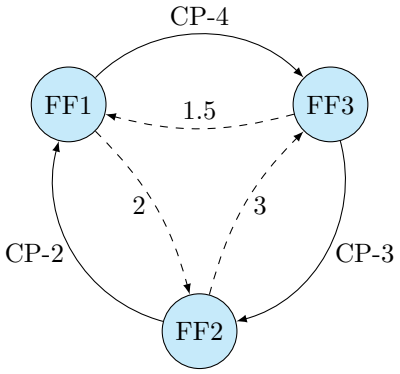


Figure 1: A timing constraint graph on three flip-flops. Solid edges encode setup relations, with weight $T_{\mathrm{CP}} - D_{ij} - T_{\mathrm{setup}}$, and dashed edges encode hold relations, with weight $d_{ij} - T_{\mathrm{hold}}$. A negative cycle certifies that no clock skew schedule satisfies all constraints and that delay padding is required.

$$T_{\mathrm{skew}}(i,j) = t_i - t_j. \tag{1}$$

Only the differences $t_i - t_j$ matter: shifting every clock by a common constant changes nothing. The setup and hold constraints bound this difference from above and below. Let D_{ij} be the maximum (setup) path delay from FF_i to FF_j, let d_{ij} be the minimum (hold) path delay, let T_{setup} and T_{hold} be the setup and hold times of the capturing flip-flop, and let T_{CP} be the clock period. Then

$$T_{\mathrm{skew}}(i,j) \le T_{\mathrm{CP}} - D_{ij} - T_{\mathrm{setup}}, \tag{2}$$
$$T_{\mathrm{skew}}(i,j) \ge T_{\mathrm{hold}} - d_{ij}. \tag{3}$$

Inequality (2) asks that the data launched at i reach j and settle before the next capturing edge; inequality (3) asks that the data not arrive so early that it corrupts the value captured at the current edge. Every data path therefore defines an admissible interval for the skew.

2.2 The timing constraint graph

A system of difference constraints $x_i - x_j \le w_{ij}$ is feasible if and only if the associated graph has no negative cycle [9], [10]. Collecting (2)–(3) over all paths yields exactly such a system, whose graph is the *timing constraint graph* (TCG) $G(V, E)$. Each vertex $i \in V$ represents a flip-flop FF_i, and a virtual vertex v_0 represents the primary inputs and outputs. A *solid* edge $e(i, j)$ encodes a setup relation and a *dashed* edge $e(j, i)$ a hold relation, each weighted by the corresponding right-hand side. Figure 1 shows a small example.

A negative cycle—a cycle whose edge weights sum to a negative value—certifies that no skew assignment satisfies all constraints simultaneously. Summing the constraints around a cycle C eliminates the potentials t_i and leaves

$$0 \le \sum_{e_{ij} \in C} \big(T_{\mathrm{CP}} - D_{ij} - T_{\mathrm{setup}}\big), \tag{4}$$
$$0 \le \sum_{e_{ij} \in C} \big(d_{ij} - T_{\mathrm{hold}}\big). \tag{5}$$

A violation of either inequality means the cycle is negative and CSS alone cannot repair the circuit. Detecting such a cycle, and the closely related parametric problems of minimizing the clock period or the cost-to-time ratio, are solved by Bellman–Ford, Lawler's parametric search, Howard's policy iteration, and Karp's

minimum-mean-cycle characterization [11], [12], [13]. Notably, (5) is the inequality that delay padding can relax: increasing the minimum path delays d_{ij} raises the right-hand side, so a sufficient amount of padding removes the negative cycle.

2.3 Delay padding and its limitations

Delay padding adds a nonnegative vector p to the path delays in the hold constraint, turning (3) into

$$T_{\text{skew}}(i,j) \geq T_{\text{hold}} - \big(d_{ij} + p_{ij}\big), \qquad p_{ij} \geq 0. \tag{6}$$

If the goal is to minimize $\sum_{ij} p_{ij}$, then (6) is the dual of a minimum-cost flow problem and is solvable by network simplex [7]. This is the conventional approach, and it has three shortcomings. First, *cost*: as noted, $\sum p$ does not reflect the true expense of a cell substitution. Second, *feasibility*: a path without a valid insertion site cannot receive padding at all, regardless of the value a flow solution assigns to it. Third, *structure*: the model treats the setup and hold paths of a register pair as independent, although in a real netlist they may share logic. We address feasibility and structure with PRA (Section 4), and cost and multi-corner consistency with dual decomposition (Section 5).

3 Statistical Timing under Process Variation

3.1 Why a Gaussian model is not enough

Statistical static timing analysis (SSTA) replaces deterministic delays with probability distributions and propagates them through the circuit to obtain a distribution of possible timing outcomes [14]. Early SSTA methods modeled delay variation as Gaussian, which is convenient because sums of Gaussians remain Gaussian and quantiles are available in closed form. At advanced nodes this assumption breaks down. The distribution of a *maximum* path delay—the quantity that governs setup—is markedly asymmetric and heavy-tailed, so a Gaussian fit captures the bulk of the distribution but misrepresents precisely the upper tail on which the timing yield depends. Because circuit delays are strictly positive, the appropriate models are the non-Gaussian, heavy-tailed ones. Yield-driven scheduling methods that exploit such models, and the specific slack-allocation rules they induce, are surveyed elsewhere [15], [16]; the present work is complementary, in that it uses the resulting delay quantiles inside a multi-corner padding formulation.

3.2 The generalized extreme value distribution

We model path delays with the *generalized extreme value* (GEV) distribution [17], which unifies the three extreme-value families and represents skewness and tail weight through three parameters: location μ, scale $\sigma > 0$, and shape ξ. Its probability density and cumulative distribution functions are

$$f(x) = \frac{1}{\sigma}\, t(x)^{\xi+1} e^{-t(x)}, \tag{7}$$

$$F(x) = e^{-t(x)}, \qquad t(x) = \Big[1 + \xi\,\frac{x-\mu}{\sigma}\Big]^{-1/\xi} \quad (\xi \neq 0). \tag{8}$$

The support of F is $[\mu - \sigma/\xi, \infty)$ for $\xi > 0$ and $(-\infty, \mu - \sigma/\xi]$ for $\xi < 0$. Because delays are positive and their maxima are heavy-tailed, the case $\xi \neq 0$ is the relevant one. For statistical timing checks we need the *quantile function*, the inverse of (8):

$$Q(\beta) = \mu + \frac{\sigma}{\xi}\Big((-\ln \beta)^{-\xi} - 1\Big), \tag{9}$$

which returns the delay value that is not exceeded with probability β.

3.3 Statistical timing constraints

Let $\tilde{D}$ and $\tilde{d}$ be the GEV-modeled maximum and minimum path delays. The deterministic constraints (2)–(3) become probabilistic: we require that the setup and hold conditions hold jointly with probability at least β. Using the quantile function (9), they admit the deterministic equivalents

$$T_{\mathrm{CP}} - T_{\mathrm{setup}} - T_{\mathrm{skew}} \geq Q_D(\beta), \tag{10}$$

$$T_{\mathrm{hold}} - T_{\mathrm{skew}} \geq Q_d(1-\beta). \tag{11}$$

The setup constraint depends on the *upper* tail $Q_D(\beta)$, which is stressed in slow corners, whereas the hold constraint depends on the *lower* tail $Q_d(1-\beta)$, stressed in fast corners. Because every CDF is monotone, Q is monotone and the yield parameter β remains a single scalar; the parametric machinery below therefore applies unchanged. This is precisely what makes the multi-corner problem concrete: each corner k (FF, SS, TT) supplies its own GEV parameters and hence its own quantiles $Q_D^k(\beta)$ and $Q_d^k(1-\beta)$.

4 Path Relationship Analysis

4.1 Feasible positions before values

The cost and feasibility problems of Section 2 share a root cause: the conventional formulation asks *how much* delay to insert before it asks *where* delay may be inserted. In a standard-cell design the second question is prior, because many paths admit no insertion site at all and the physical operation is a cell swap rather than the addition of an abstract delay. We therefore invert the order of decisions: PRA first finds the physically realizable positions, and only then are the delay values computed. This is both more faithful to the physical design flow and more reliable, because it never proposes a solution that cannot be manufactured.

The analysis examines, for each register pair (i, j), the structural relationship between the maximum-delay (setup) route and the minimum-delay (hold) route through the combinational logic that lies between the two registers. Their degree of overlap determines which padding variables are free, which are coupled, and how the TCG must be modified. Four configurations cover all cases.

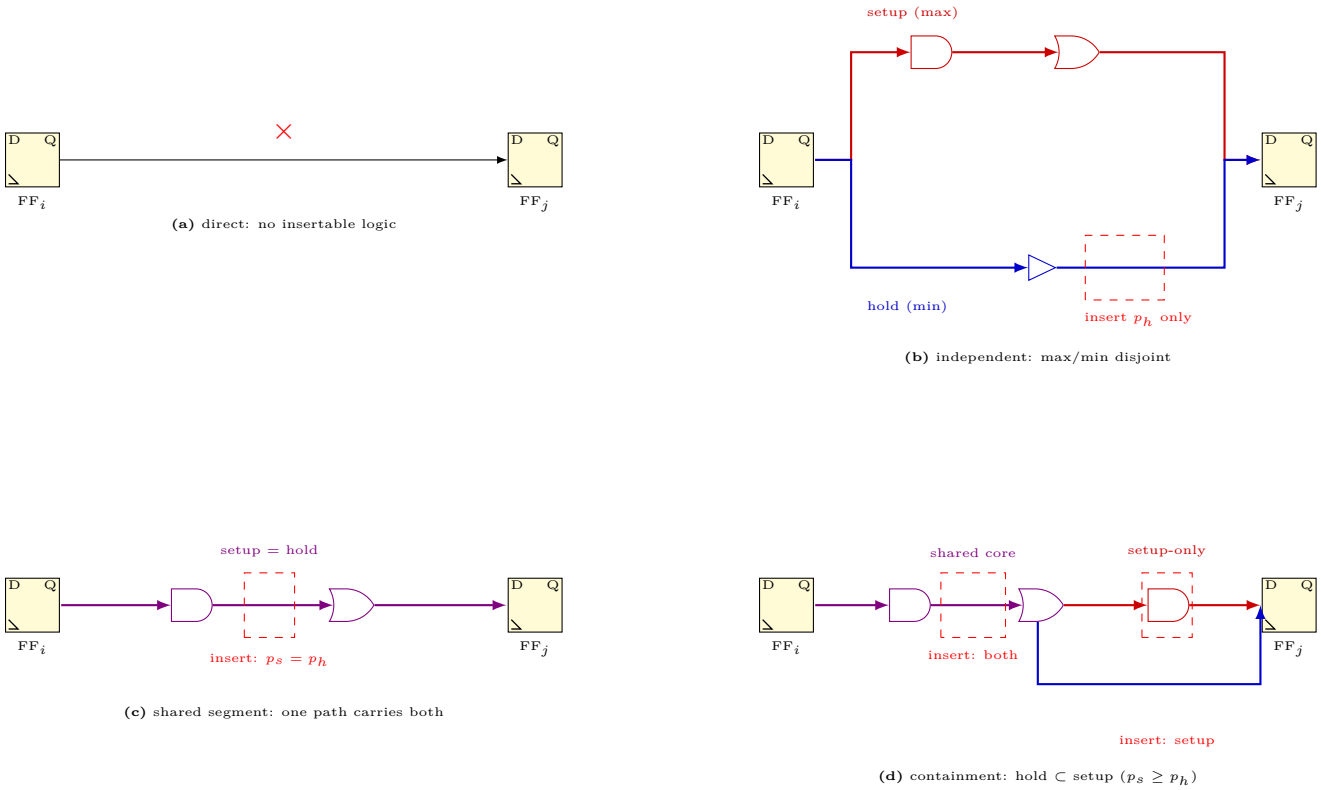
4.2 The four structural configurations

The four configurations are summarized in Table 1 and illustrated in Figure 2.

Table 1: The four structural configurations of path relationship analysis and the constraint each imposes on the setup padding p_s and hold padding p_h.

Type	Structure	Padding constraint	TCG modification
(a)	direct connection, no insertable logic	none (no insertion)	unchanged
(b)	setup and hold routes disjoint	p_s, p_h independent	auxiliary nodes u_s, u_h
(c)	one route carries both delays	$p_s = p_h$	single shared node
(d)	hold route nested in setup route	$p_s \geq p_h$	coupling edge $u_s \to u_h$

Type (a), *direct connection*: the flip-flop pair is wired directly, with no combinational logic and hence no insertion site; the TCG is left unchanged. Type (b), *independent structure*: the maximum- and minimum-delay routes are disjoint (they share only the register endpoints), so a delay inserted on the hold route raises d_{ij} without touching D_{ij}; the paddings p_s and p_h are therefore independent. Type (c), *one route carries both*



(a) direct: no insertable logic

(b) independent: max/min disjoint

(c) shared segment: one path carries both

(d) containment: hold $\subset$ setup ($p_s \geq p_h$)

Figure 2: The four structural configurations of path relationship analysis, drawn as gate-level circuits. Red edges are maximum-delay (setup) routes, blue edges are minimum-delay (hold) routes, and violet edges are routes that carry both. Red dashed boxes mark where delay may be inserted. (a) direct connection: no insertable logic. (b) independent: disjoint routes, so p_h is inserted on the hold route without affecting setup. (c) one route carries both, so $p_s = p_h$. (d) containment: the hold route is a sub-path of the setup route, so a shared insertion raises both delays and a setup-only insertion raises only D, giving $p_s \geq p_h$.

delays: a single route supplies both the maximum and the minimum delay, so any inserted delay raises D_{ij} and d_{ij} by the same amount and the paddings must coincide, $p_s = p_h$. Type (d), *hierarchical containment*: the hold route is a sub-path of the setup route, so a delay on the shared core raises both delays whereas a delay on the setup-only tail raises D_{ij} alone, giving $p_s \geq p_h$.

The distinction has a direct consequence for the objective. Only in configuration (b) does padding the hold route increase d_{ij} while leaving D_{ij} unchanged, which is exactly what shortens the critical cycle and lowers the clock period; in (c) and (d) a shared insertion raises D_{ij} and d_{ij} together and the period is unchanged. Path relationship analysis therefore tells the optimizer not merely *where* delay can physically be inserted, but whether inserting it can help at all. The underlying admissibility condition—delay may be added at a node only if no long path through that node is pushed past its bound—is classical [4], and the coupling produced by overlapping routes is the reconvergent-path limitation identified by Taskin and Kourtev [18].

4.3 TCG enhancement with auxiliary nodes

PRA is realized on the graph by introducing nonnegative auxiliary potentials u_s and u_h. The padding values are the differences between these potentials and the originating register potential,

$$p_s = u_s - u_i \geq 0, \qquad p_h = u_h - u_i \geq 0. \tag{12}$$

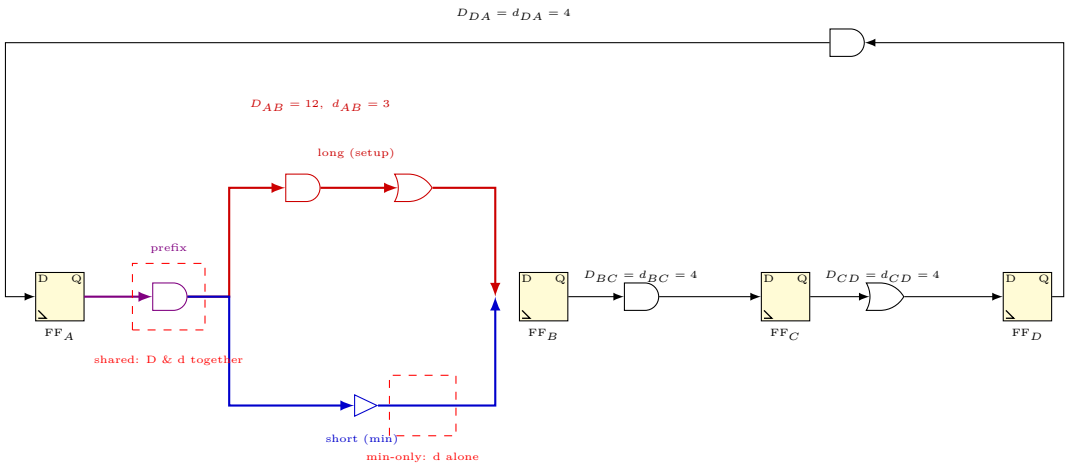


Figure 3: Circuit of the worked example: a ring of four flip-flops. Red edges are maximum-delay (setup) routes, blue edges are minimum-delay (hold) routes, and violet is the segment shared by both. The data path $A \to B$ is reconvergent: the red dashed insertion point on the shared prefix moves D_{AB} and d_{AB} together, while the one on the short min-only branch moves d_{AB} alone. The simple paths $B \to C$, $C \to D$, and $D \to A$ have $D = d$, so no insertion can separate their two delays.

The structural configuration then fixes the coupling between u_s and u_h: no auxiliary edge for type (b), $u_s = u_h$ for type (c), and $u_s \geq u_h$ for type (d), the last encoded by a zero-weight edge from u_s to u_h. With these additions, the question "can the timing be repaired by padding?" is again a feasibility problem on a modified graph, so the negative-cycle test of Section 2 applies unchanged. The padding values are potentials on the auxiliary edges, which means the entire network-potential machinery carries over without modification. The result is precisely the "determine the position first, then the value" strategy: positions that are structurally infeasible are pruned before any optimizer sees them, and the coupling constraints prevent the optimizer from proposing physically contradictory paddings.

4.4 Worked example: where and how much to pad

The following deterministic example (no process variation) isolates delay padding and path relationship analysis. Figure 3 shows a ring of four flip-flops A, B, C, D together with the combinational logic between them, and Figure 4 shows the corresponding timing constraint graph. The data path from FF_A to FF_B is *reconvergent*: a shared prefix fans out into a long branch and a short branch, so the same logic block has a maximum delay $D_{AB} = 12$ and a minimum delay $d_{AB} = 3$. The remaining data paths are simple, with $D_{BC} = d_{BC} = 4$, $D_{CD} = d_{CD} = 4$, and $D_{DA} = d_{DA} = 4$. Solving for the minimum feasible clock period returns 9.00, attained by the max/min pair $D_{AB} - d_{AB} = 12 - 3$ of the $\mathrm{FF}_A \to \mathrm{FF}_B$ data path.

Now consider the two kinds of insertion. If delay is added to a segment that lies on **both** the long and the short branch—the shared prefix, for instance—then D_{AB} and d_{AB} rise by the same amount, their difference is unchanged, and the period does not move. The same holds for every simple path, whose maximum and minimum delays are equal: inserting delay there cannot separate them, so a one- or two-gate path can never be helped by padding. If instead delay is added to a segment that lies on the **short branch alone**, then d_{AB} rises while D_{AB} stays, their difference shrinks, and the period falls. With 3 units of padding on the min-only branch the period drops from 9.00 to 6.00, a 33.3% improvement. Figure 5 shows both behaviors.

The example makes the mechanism concrete. Padding helps exactly when it moves d_{ij} without moving D_{ij}; because the two delays belong to the *same* local data path, this is possible only where the minimum-delay route and the maximum-delay route differ—only on a reconvergent path, and only on a segment of it that misses every maximum-delay route. That is precisely what path relationship analysis identifies, and it is why the four configurations of Section 4 carry information that a plain min-cost-flow formulation does not. The period cannot fall below the setup-only bound $(D_{AB} + D_{BC} + D_{CD} + D_{DA})/4 = 6.00$, which no amount

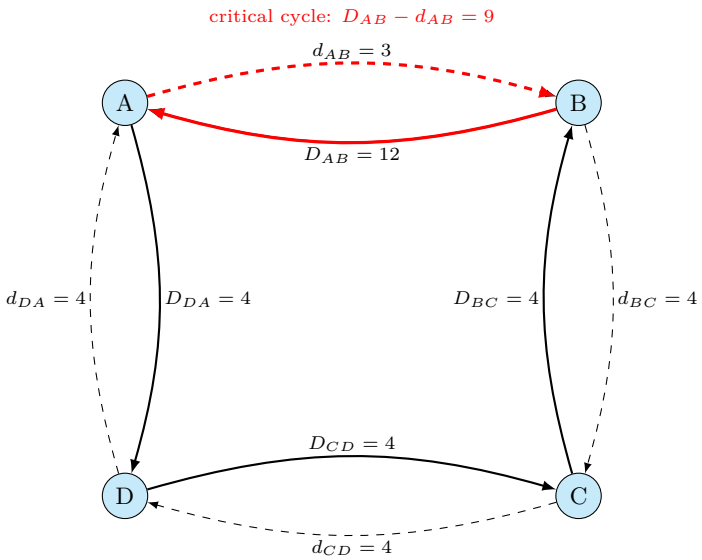


Figure 4: Timing constraint graph of the worked example. Solid arcs are setup (maximum-delay) constraints and dashed arcs are hold (minimum-delay) constraints. The red cycle joins the hold arc d_{AB} and the setup arc D_{AB} of the same local data path $A \to B$; its weight $D_{AB} - d_{AB} = 9$ is the binding bound.

of hold padding can reduce, so padding is effective only up to the point at which the setup structure takes over. The program `examples/delay_padding.py` in the `digraphx` repository reproduces every number in this section.

5 Dual Decomposition for Multi-Corner Delay Padding

5.1 Multi-corner consistency and the ping-pong effect

Under multi-corner timing analysis, each corner k (for example FF, SS, or TT) has its own delays and therefore its own constraints. Padding, however, is realized once in silicon and must be *consistent* across corners: the same physical cell substitution must be admissible in every corner. Solving the corners independently produces per-corner optima that generally disagree, and iterating between them is exactly the ping-pong behavior described in Section 1. The correct objective is therefore a *single* padding assignment that is feasible for all corners simultaneously.

5.2 Lagrangian formulation

Let $y_k \in \mathbb{R}^n$ collect the per-corner adjusted edge delays (the skews induced by scheduling and padding in corner k), and let $y_{\text{shared}} \in \mathbb{R}^n$ be the common target. Coupling the corners through y_{shared} yields the problem

$$\min_{\{y_k\}} \quad \sum_k \lambda_k^\top \left(y_k - y_{\text{shared}}\right) \tag{13}$$

$$\text{s.t.} \quad Au = y, \tag{14}$$

$$y_{k,ij} \le T_{\text{CP}} - D_{k,ij} - T_{\text{setup}}, \quad \forall (i,j),\ \forall k, \tag{15}$$

$$y_{k,ij} \ge T_{\text{hold}} - d_{k,ij}, \quad \forall (i,j),\ \forall k, \tag{16}$$

where A is the node-edge incidence matrix of the TCG, u is the vector of potentials, and λ_k are the multipliers that penalize deviation of corner k from the shared target. The term $\lambda_k^\top (y_k - y_{\text{shared}})$ is a *soft* consistency

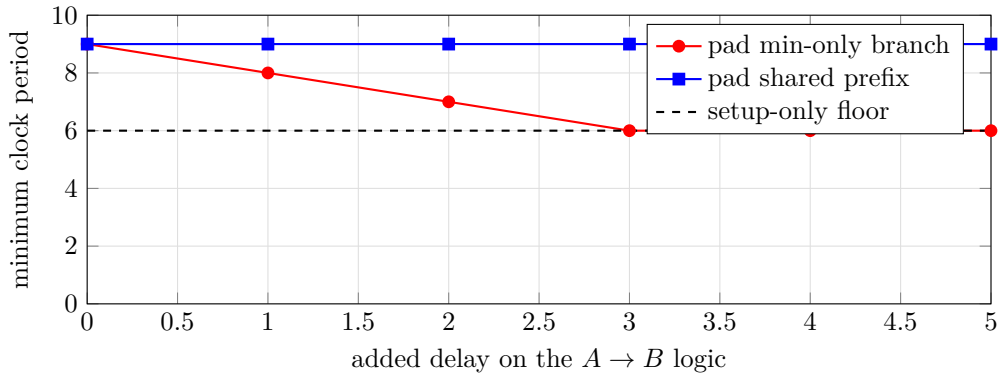


Figure 5: Minimum feasible clock period as a function of the delay added to the $A \to B$ logic. Padding the min-only branch (red) lowers the period from 9.00 to 6.00; padding the shared prefix (blue) raises D_{AB} and d_{AB} together and leaves the period at 9.00. The dashed line is the setup-only bound $(D_{AB}+D_{BC}+D_{CD}+D_{DA})/4 = 6.00$.

penalty: it drives the local solutions toward a common value without forcing equality at every iteration.

5.3 The iterative algorithm

Lagrangian relaxation decouples the consistency requirement. At each iteration the shared target and the multipliers are held fixed, and the corners are solved independently and in parallel.

1. *Solve the subproblems.* For each k, minimize the local objective subject to (14)–(16), treating y_{shared} and λ_k as constants.
2. *Update the shared target* as the average of the local solutions, (17), where K is the number of corners.
3. *Update the multipliers* by subgradient ascent, (18), with step size $\rho > 0$ controlling the convergence rate.

The two update rules are, respectively,

$$y_{\text{shared}} \leftarrow \frac{1}{K} \sum_{k=1}^{K} y_k, \tag{17}$$

$$\lambda_k \leftarrow \lambda_k + \rho \left(y_k - y_{\text{shared}} \right), \tag{18}$$

Iterations continue until the local solutions agree with the shared target, that is, until $\|y_k - y_{\text{shared}}\| \to 0$. At convergence a single padding assignment satisfies every corner. This is precisely the consistency that per-corner iteration fails to achieve: the oscillation is removed because no corner is ever solved in isolation without reference to the shared target, and the average step (17) acts as the consensus mechanism while (18) accumulates the pricing information that pulls the corners together. Each subproblem is a network-flow (minimum-cost potential) problem and is solved by convex optimization [19], [20]; the averaging and subgradient steps are standard Lagrangian procedures [21], [22], [23].

5.4 Statistical multi-corner formulation

Replacing the deterministic delays with GEV quantiles makes the subproblems variation-aware. For corner k, the constraints (15)–(16) become

$$y_{k,ij} \le T_{\text{CP}} - Q_D^k(\beta) - T_{\text{setup}}, \tag{19}$$

$$y_{k,ij} \ge T_{\text{hold}} - Q_d^k(1-\beta), \tag{20}$$

Algorithm 1: Multi-corner delay padding via binary search and dual decomposition

Input : netlist; corners $k = 1, \ldots, K$; yield target β; tolerance ϵ
Output: minimum feasible clock period $T^{\star}_{\mathrm{CP}}$; padding $p^{\star}$

$T_{\mathrm{low}} \leftarrow$ lower bound; $T_{\mathrm{high}} \leftarrow$ upper bound;
while $T_{\mathrm{high}} - T_{\mathrm{low}} > \epsilon$ **do**
  $T_{\mathrm{CP}} \leftarrow (T_{\mathrm{high}} + T_{\mathrm{low}})/2$;
  run GEV-based multi-corner SSTA; extract $Q^k_D(\beta)$ and $Q^k_d(1-\beta)$;
  run PRA; enhance the TCG with auxiliary nodes and coupling edges;
  if *the enhanced TCG has a negative cycle* **then**
    $T_{\mathrm{low}} \leftarrow T_{\mathrm{CP}}$; `// infeasible`
  else
    solve the per-corner subproblems, coordinating via (17)–(18) to convergence;
    if *a consistent feasible padding is found* **then**
      $T_{\mathrm{high}} \leftarrow T_{\mathrm{CP}}$;
      record $p^{\star} \leftarrow p$;
    else
      $T_{\mathrm{low}} \leftarrow T_{\mathrm{CP}}$;
return $T^{\star}_{\mathrm{CP}} = T_{\mathrm{high}}$, $p^{\star}$

with the objective (13) unchanged. The single scalar β is shared by all corners, so the dual decomposition coordinates not only the delay values but also the statistical guarantee: the returned padding meets the target timing yield simultaneously in every corner.

6 Overall Timing Optimization Flow

The pieces combine into a flow that finds the minimum clock period for which a consistent, physically feasible padding exists. The outer loop is a binary search on T_{CP} between a lower bound T_{low} and an upper bound T_{high}. For a candidate period the flow performs GEV-based multi-corner SSTA, extracts the quantile delays, runs PRA to identify feasible positions, enhances the TCG, and tests for negative cycles. If a negative cycle persists after the padding variables are fixed, the candidate period is infeasible and T_{low} is raised; otherwise the dual decomposition solver computes the corner-aware padding, the period is feasible, and T_{high} is lowered. The search terminates when $T_{\mathrm{high}} - T_{\mathrm{low}} < \epsilon$. Algorithm 1 states the procedure.

The negative-cycle test plays two roles. It is first a *feasibility certificate*: a negative cycle means that, even after every physically available padding position has been used, the constraints cannot be met, so the period must be increased. It is second a *diagnostic*: the cycle identifies the set of paths that determine the current period and therefore guides where additional padding capacity or a smaller setup delay would help most. Once the period is feasible, the dual decomposition solver returns an explicit, corner-consistent padding assignment; the binary search then tightens the interval until it is smaller than the tolerance.

7 Experimental Setup and Results

7.1 Setup

We evaluate the method on two benchmark families: the OpenCore designs [24] and the OpenABC-D dataset [25]. The technology is the open-source ASAP7 7,nm predictive process design kit [8], which provides standard cells characterized across multiple corners. We use the fast-fast (FF), slow-slow (SS), and typical-typical (TT) corners, which together capture a broad range of variation. Statistical path delay distributions are obtained from GEV-based SSTA, and all timing checks use a target yield of $\beta = 0.99$. The optimization proceeds by binary search over candidate clock periods: clock skew scheduling is applied first, and if negative cycles

persist the dual decomposition padding solver is invoked. Each per-corner subproblem is a minimum-cost potential problem solved by convex optimization with CVXPY [20].

7.2 Distribution of path relationships

Applying PRA to all benchmark paths and classifying every register pair into the four types yields two observations. First, the proportions vary widely across benchmarks, so no single fixed heuristic covers every design. Second, and more importantly, a substantial fraction of register pairs exhibit structural dependence between their maximum- and minimum-delay paths, and the *critical* paths that set the clock period tend to be of type (c) (shared path segment) or type (d) (hierarchical containment). Consequently, naively padding a critical path frequently perturbs the hold constraint of the same pair, because the two paths share logic. This validates the central tenet of PRA: positions must be identified, and their coupling respected, before values are assigned. Methods that treat setup and hold paths as independent will either over-pad or under-pad these pairs.

7.3 Delay padding performance

Table 2 reports the reduction of the optimized clock period relative to the single worst-case (SS) corner.

Table 2: Reduction of the optimized clock period relative to the single worst-case (SS) corner for the OpenCore and OpenABC-D benchmarks at a timing yield of 0.99.

Benchmark	Clock-period reduction vs. SS corner (%)
aes	4.97
gcd	9.22
ethmac	7.85
ibex	3.81
jpeg	4.80
tv80	6.88
Average	**6.12**

Three conclusions follow. First, the optimized clock period consistently lies between the best-case (FF) and worst-case (SS) single-corner results, so the method achieves a balanced trade-off across corners instead of the pessimistic SS bound. Second, the average reduction is 6.12%, with a largest gain of 9.22% on `gcd` and a smallest gain of 3.81% on `ibex`; the method therefore helps every design, though by an amount that reflects how much structural coupling and corner disagreement the design contains. Third, the largest gains coincide with the designs whose critical paths are most often type (c) or type (d), which is consistent with the PRA finding: the more the setup and hold paths interact, the more the conventional independent treatment leaves on the table, and the more the coordinated treatment recovers.

8 Discussion and Conclusion

We proposed a comprehensive timing-optimization method that integrates clock skew scheduling and delay padding under multi-corner variation. Its key contributions are three. *GEV-based statistical modeling* characterizes path delay distributions accurately, enabling robust timing-yield satisfaction without a Gaussian assumption. *Path relationship analysis* identifies viable and physically feasible padding locations by understanding setup–hold path interactions, so that delay is never proposed where it cannot be inserted and coupled paddings are never assigned contradictory values. *Dual decomposition* provides a scalable way to jointly optimize padding across corners, coordinating shared delay variables through Lagrange multipliers and thereby eliminating the ping-pong effect of corner-by-corner iteration. On the OpenCore and OpenABC-D benchmarks with the ASAP7 library, the method reduces the minimum feasible clock period by an average of 6.12% relative to the single worst-case corner while maintaining a timing yield of 0.99 in all corners.

Several limitations delimit the scope of this work and point to future directions. The padding p is treated as a continuous, nonnegative vector, whereas in a standard-cell flow delay is inserted from a *discrete* cell library, so the continuous relaxation is best read as a lower bound on the achievable period and as a guide to which paths are worth padding. PRA narrows the gap by pre-selecting physically realizable positions and by coupling the setup and hold paddings, but it does not make the values discrete. Corner correlation is also not modeled explicitly: the subproblems share the delay variables but are otherwise solved independently, so a correlated variation model could tighten the result further. Finally, the subgradient step size ρ affects the convergence rate and deserves an adaptive or safeguarded choice. We regard these as opportunities rather than obstacles, and we expect the method to compose naturally with placement and clock-tree synthesis, and to scale to larger industrial designs.

The reference implementations of the underlying graph algorithms, including negative-cycle detection and minimum-cost-to-time-ratio solvers, are available in the sibling libraries `digraphx` and `netoptim` [26], [27], and the broader formulations are developed in the accompanying lecture notes [28].

References

[1] J. P. Fishburn, "Clock skew optimization," *IEEE Transactions on Computers*, vol. 39, no. 7, pp. 945–951, 1990.

[2] J. L. Neves and E. G. Friedman, "Optimal clock skew scheduling tolerant to process variations," in *Proceedings of the 33rd annual design automation conference (DAC)*, 1996, pp. 623–628. doi: 10.1145/240518.240636.

[3] I. S. Kourtev and E. G. Friedman, *Timing optimization through clock-skew scheduling*. Boston, MA: Springer, 2000.

[4] N. V. Shenoy, R. K. Brayton, and A. L. Sangiovanni-Vincentelli, "Minimum padding to satisfy short path constraints," in *Proceedings of the IEEE/ACM international conference on computer-aided design (ICCAD)*, 1993, pp. 156–161. doi: 10.1109/ICCAD.1993.580048.

[5] C. Albrecht, B. Korte, J. Schietke, and J. Vygen, "Cycle time and slack optimization for VLSI-chips," in *Proceedings of the IEEE/ACM international conference on computer-aided design (ICCAD)*, 1999, pp. 232–238. doi: 10.1109/ICCAD.1999.810654.

[6] S. Held, B. Korte, J. Mäßberg, M. Ringe, and J. Vygen, "Clock scheduling and clock tree construction for high performance ASICs," in *Proceedings of the IEEE/ACM international conference on computer-aided design (ICCAD)*, 2003, pp. 232–239.

[7] R. K. Ahuja, T. L. Magnanti, and J. B. Orlin, *Network flows: Theory, algorithms, and applications*. Englewood Cliffs, NJ: Prentice Hall, 1993.

[8] L. T. Clark *et al.*, "ASAP7: A 7-nm FinFET predictive process design kit," *Microelectronics Journal*, vol. 53, pp. 105–110, 2016.

[9] R. Bellman, "On a routing problem," *Quarterly of Applied Mathematics*, vol. 16, no. 1, pp. 87–90, 1958.

[10] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.

[11] E. L. Lawler, *Combinatorial optimization: Networks and matroids*. New York: Holt, Rinehart; Winston, 1976.

[12] R. A. Howard, *Dynamic programming and markov processes*. Cambridge, MA: MIT Press, 1960.

[13] R. M. Karp, "A characterization of the minimum cycle mean in a digraph," *Discrete Mathematics*, vol. 23, no. 3, pp. 309–311, 1978.

[14] C. Visweswariah, K. Ravindran, K. Kalafala, S. G. Walker, and S. Narayan, "First-order incremental block-based statistical timing analysis," in *Proceedings of the 41st annual design automation conference (DAC)*, 2004, pp. 331–336.

[15] J.-L. Tsai, D. H. Baik, C. C.-P. Chen, and K. K. Saluja, "Yield-driven, false-path-aware clock skew scheduling," in *IEEE design & test of computers*, 2005, pp. 214–222.

[16] X. Wei, Y. Cai, and X. Hong, "Clock skew scheduling under process variations," in *Proceedings of the 7th international symposium on quality electronic design (ISQED)*, 2006, pp. 237–242.

[17] A. F. Jenkinson, "The frequency distribution of the annual maximum (or minimum) values of meteorological elements," *Quarterly Journal of the Royal Meteorological Society*, vol. 81, no. 348, pp. 158–171, 1955.

[18] B. Taşkın and I. S. Kourtev, "Delay insertion method in clock skew scheduling," in *Proceedings of the 2005 international symposium on physical design (ISPD)*, 2005, pp. 47–54. doi: 10.1145/1055137.1055149.

[19] S. Boyd and L. Vandenberghe, *Convex optimization*. Cambridge, UK: Cambridge University Press, 2004.

[20] S. Diamond and S. Boyd, "CVXPY: A Python-embedded modeling language for convex optimization," in *Journal of machine learning research*, 2016, pp. 1–5.

[21] H. Everett, "Generalized lagrange multiplier method for solving problems of optimum allocation of resources," in *Operations research*, 1963, pp. 399–417.

[22] D. P. Bertsekas, *Nonlinear programming*, 2nd ed. Belmont, MA: Athena Scientific, 1999.

[23] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, "Distributed optimization and statistical learning via the alternating direction method of multipliers," *Foundations and Trends in Machine Learning*, vol. 3, no. 1, pp. 1–122, 2011.

[24] OpenCores, "OpenCores: Open hardware IP cores and Verilog benchmarks." https://opencores.org, 2026.

[25] A. B. Chowdhury, B. Tan, R. Karri, and S. Garg, "OpenABC-D: A large-scale dataset for machine learning guided integrated circuit synthesis," *arXiv preprint arXiv:2110.11292*, 2021.

[26] W.-S. Luk, "digraphx: Directed-graph algorithms for network optimization." Reference implementations in Python, C++, and Rust (siblings `digraphx`, `digraphx-cpp`, `digraphx-rs`), https://github.com/luk036/digraphx, 2026.

[27] W.-S. Luk, "netoptim: Network-flow optimization oracles and cutting-plane solvers." Reference implementations in Python, C++, and Rust (siblings `netoptim`, `netoptim-cpp`, `netoptim-rs`), https://github.com/luk036/netoptim, 2026.

[28] W.-S. Luk, "Algorithms for design-for-manufacturability and network optimization: Lecture notes." Online course notes, https://luk036.github.io, 2026.